



Section 1 / 11

Buffer Overflow

Binary exploitation is among the most essential skills for any pentester. It is usually the way to find the most advanced vulnerabilities in programs and operating systems and requires a lot of skill. Over the years, many protections have been added to the way memory is handled by the OS Kernel and how binaries are compiled to prevent such vulnerabilities. Still, there are always newer ways to exploit minor mistakes found in binaries and utilize them to gain control over a remote machine or gain higher privilege over a local machine.

As binary and memory protections become more advanced, however, so do the binary exploitation methods. This is why modern binary exploitation methods require a deep understanding of Assembly language, Computer Architecture, and the fundamentals of binary exploitation.

Both Assembly language and Computer Architecture were thoroughly covered in the [Intro to Assembly Language](#) module, and the [Stack-Based Buffer Overflows on Linux x86](#) module also covered basics of binary exploitation on Linux.

Buffer Overflows

In Binary exploitation, our primary goal is to subvert the binary's execution in a way that benefits us. Buffer Overflows are the most common type of binary exploitation, but other types of binary

exploitation exist, such as **Format String** exploitation and **Heap Exploitation**.

A buffer overflow occurs when a program receives data that is longer than expected, such that it overwrites the entire buffer memory space on the **stack**. This can overwrite the next Instruction Pointer **EIP** (or **RIP** in *x86_64*), which causes the program to crash because it will attempt to execute instructions at an invalid memory address. By forcing the program to crash, this is the most basic example of exploiting buffer overflows - known as a Denial of Service (**DOS**) attack.

Another basic attack is to overwrite a value on the stack to change the program's behavior. For example, if an exam program had a buffer overflow vulnerability, we can overwrite the buffer enough to overwrite our score. Since our exam score is stored in the stack in this example, we could take advantage of this flaw to change our score.

If we are a bit more sophisticated, we can change the address of **EIP** to an instruction that will execute our shellcode. This would allow us to execute any command we want instead of just crashing the program, known as Jumping to Shellcode.

With more advanced memory protections, it may not be possible to load our entire shellcode and point to it. Instead, we may use a combination of instructions from the binary to execute a particular function and overwrite various pointers to change the program execution flow. This is known as Return Oriented Programming (**ROP**) attacks.

Finally, modern programs and operating systems may use the Heap instead of the Stack to store buffer memory, which would require Heap Overflows or Heap Exploitation methods.

Stack Overflow

Let's start by demonstrating how the stack works in storing data. The stack has a Last-in, First-out (LIFO) design, which means we can only `pop` out the last element `push` ed into the stack. If we `push` an item into the stack, it would be located on the top of the stack. If we `pop` anything from the stack, the item located at the top of the stack would get popped.

The following table demonstrates how the stack works. We can click on `push` to push a value from `eax` to the stack, and `pop` to pop the top value from the stack into `eax` :

0xabcdef	<-- Top of Stack (<code>\$esp</code>)
0x12345678	<-- Bottom of Stack (<code>\$ebp</code>)

eax:

push

pop

Reset Stack

The above example correctly receives buffer data, such that it never gets overflowed to the next item. Now let's review another example that does not correctly store data on the stack.

The following example expects an input from us that is eight characters long. But what would happen if we sent something longer? Let's try to send '01234567890123456789':

0xabcdef	<-- Top of Stack (\$esp)
0x401000	<-- Return Address (\$eip)
0x12345678	<-- Bottom of Stack (\$ebp)

eax:

push

pop

Reset
Stack

As we can see, when we send a string that is longer than expected, it overwrites other existing values on the stack and would even overwrite the entire stack if it is long enough. Most importantly, we see that it overwrote the value at EIP, and when the function tries to return to this address, the program will crash since this address '0x6789' does not exist in memory. This

happens because of the LIFO design of the stack, which **grows upwards**, while **a long string** overflows values **downwards** until it eventually overwrites the return address **EIP** and the bottom of the stack pointer **EBP**. This was explained in the **Intro to Assembly Language** module.

Whenever a function is called, a new stack frame is created, and the old **EIP** address gets pushed to the top of the new stack frame, so the program knows where to return once the function is finished. For example, if our buffer input overwrites the entire stack and return address **EIP**, then the overwritten **EIP** address will be called when the function returns due to a **RET** instruction.

If we calculate our input precisely, we can place a valid address in the location where **EIP** is stored. This would lead the program to go to our overwritten address when it returns and subvert the program execution flow to an address of our choosing.

Real-World Examples

There have been numerous incidents where stack overflow exploits were used to break into restricted systems, like mobile phones or gaming consoles.

In 2010, iPhones running on iOS 4 were jailbroken using the **greenpois0n** jailbreak, which utilized two different exploits to gain kernel-level access over the iPhone and install unofficial/unsigned software and apps. One of these exploits was a stack-based buffer overflow on the iPhone's **HFS Volume Name**. At that time, iPhones did not automatically randomize the address space, and iOS 4.3 patched these vulnerabilities and introduced memory protections like randomizing address spaces with Address Space Layout Randomization (**ASLR**).

A stack-based buffer overflow exploit was also used to gain kernel-level access on the original PlayStation Portable (PSP) running Firmware v2.0. This allowed the use of pirated games as well

as installing unsigned software. The **TIFF Exploit** exploits a vulnerability found in the TIFF image library used in the PSP's photo viewer. This leads to code execution by simply viewing a malicious **.tiff** file in the photo viewer after setting the background to a corrupt **.png** image. Another similar stack overflow exploit was later discovered in the PSP game "Grand Theft Auto: Liberty City Stories", which had an overflow vulnerability in its Saved Game data and can be exploited by loading a malicious load file.

Another example of a stack-based buffer overflow exploit was used to gain kernel-level access on the original Nintendo Wii, which also allowed the use of pirated games and the installation of unsigned software. The **Twilight Hack** exploits a vulnerability found in "The Legend of Zelda: Twilight Princess" game and is also exploited by loading malicious Saved Game data, by using a long name for Link's horse "Epona".

Finally, in 2020 a new vulnerability was found for the PlayStation 2, almost 20 years after its initial release. The **FreeDVDBoot** exploits a vulnerability in the PS2's DVD player by placing a malicious "VIDEO_TS.IFO" file. This gets read by the DVD player and causes an overflow that can lead to code execution. This was the first-ever PS2 hack that is entirely software-based, as all older hacks utilized some form of hardware like a malicious memory card to load and execute unsigned software.

Of course, operating systems like Windows, Linux, and macOS were always the first target for stack-based buffer overflow exploits. There have been numerous such vulnerabilities found in all of these systems and software running on them. By detecting these vulnerabilities before products go into production, we would reduce the occurrence of potentially catastrophic pitfalls.

Stack Overflow Protections

As we may notice from the above examples, most of them are pretty old, aging back at least a decade. This is because modern operating systems have many protections for the stack, like preventing code execution or randomly changing the memory addresses. These protections make it so we cannot easily run our code placed on the stack or pre-calculate the memory address to jump to.

However, even with these types of protections, if a program is vulnerable to a Buffer Overflow, there are advanced methods to bypass these protections. Some examples include the previously mentioned Return Oriented Programming ([ROP](#)) or Windows-specific exploitation methods like Egg Hunting or Structured Exception Handling ([SEH](#)) exploitation.

Furthermore, modern compilers prevent the usage of functions that are vulnerable to Stack overflows, which significantly reduces the occurrence of stack-based buffer overflows. This is why stack-based buffer overflows are less common these days. At the same time, other more advanced types of binary exploitation are more common, as they can't be mitigated by simply enabling a protection method.

Why Learn Basic Stack Overflows?

In this module, we'll learn how to gain code execution through basic stack-based buffer overflows. We will do so on applications and systems that do not have any memory protection. Otherwise, we'd require more advanced methods to gain code execution.

But if basic stack-based overflows are no longer common these days, then why should we learn them?

We do so because learning them gives us a good understanding of the basics of binary exploitation and the fundamentals of exploit development.

Furthermore, once we master how to detect and exploit basic stack-based buffer overflows, it will be much easier to learn **Structured Exception Handling (SEH)**, which is very common in modern Windows systems.

Finally, once we get a good grip on basic stack overflows and basic mitigation bypasses, we would be ready to start learning advanced mitigation bypass methods, like (**ROP**) and other advanced binary exploitation methods.

Section 1 / 11

 +20

Next

Mark Complete & Next